SE Lab 4: Vibe Coding (Battleship vs AI)

Name: Venkata Srivaibhav Nukaraju

SRN: PES2UG24CS579

Section: 5 J

Course: UE24CS341A Software Engineering

1. Links

Item	Link
Assigned repo	https://github.com/SETAPESU26/19_battleship
GitHub repo (submission)	https://github.com/Redslayer112/SELAB/tree/main/Lab-4
ChatGPT chat history	https://chatgpt.com/share/6ad07296-98d0-83ec-a4dd-e59007d04197

2. Defects and Changes

The original AI returns 1-based strings, and the game reads them as 0-based tuples. Every AI shot lands one row and one column off, so the AI reports false hits. The ship counter subtracts the wrong shot set, so it never drops. The game loop stops after the AI shot and never records it on the player board.

Prompt	Task	Change
1	Task 1: coordinates	AI returns 0-based tuples; AI shots hit the player board; counter and loss check fixed
2	Task 2: fleet and win	Three ships per side (3, 2, 2); per-ship damage; sunk messages; game ends when a fleet sinks
3	Tasks 3 and 4: AI and feedback	AI targets untried neighbours of hits; returns None on a full board; one message per real shot

3. Updated Code

main.py

```
from game import Battleship

if __name__ == "__main__":
    Battleship().run()
```

game.py

```
from board import Board
from ai import AI


class Battleship:
    def __init__(self):
        self.player = Board()
        self.enemy = Board()
        self.ai = AI()
        self._setup()

    def _setup(self):
        # All positions are 0-based (row, col).

        # Player fleet: sizes 3, 2, 2.
        self.player.place_ship({(0, 0), (0, 1), (0, 2)})
        self.player.place_ship({(2, 0), (2, 1)})
        self.player.place_ship({(4, 3), (4, 4)})

        # Enemy fleet: sizes 3, 2, 2.
        self.enemy.place_ship({(1, 1), (1, 2), (1, 3)})
        self.enemy.place_ship({(3, 0), (3, 1)})
        self.enemy.place_ship({(5, 3), (5, 4)})

    def show(self):
        print("\nYour shots are coordinates like 2,3.")
        print("Enemy ship cells remaining:", self.enemy.remaining_cells())

    def _report_shot(self, result, shooter):
        status, sunk_size = result

        if status == "miss":
            print(f"{shooter}: MISS!")
        elif status == "hit":
            print(f"{shooter}: HIT!")
        elif status == "sunk":
            print(f"{shooter} sank a ship of size {sunk_size}.")
        elif status == "repeat":
            print("Already fired there.")

    def run(self):
        print("Battleship")

        while True:
            self.show()
            raw = input("> ").strip().lower()
```

```
            if raw == "q":
                return

            try:
                r, c = map(int, raw.split(","))
                pos = (r - 1, c - 1)
            except ValueError:
                print("Use row,col.")
                continue

            if not (
                0 <= pos[0] < Board.SIZE
                and 0 <= pos[1] < Board.SIZE
            ):
                print("Outside board.")
                continue

            if pos in self.enemy.shots:
                print("Already fired there.")
                continue

            # Player's shot: resolve and report exactly once.
            result = self.enemy.fire(pos)
            self._report_shot(result, "You")

            if self.enemy.all_sunk():
                print("You sank the entire enemy fleet. You win!")
                return

            # AI chooses a coordinate without printing anything.
            ai_pos = self.ai.choose()

            if ai_pos is None:
                print("AI has no remaining shots.")
                continue

            print(f"AI fired at {ai_pos[0] + 1},{ai_pos[1] + 1}")

            # Resolve the actual shot before informing the AI.
            result = self.player.fire(ai_pos)
            self.ai.observe(ai_pos, result)

            # Only game.py reports the result.
            self._report_shot(result, "AI")

            if self.player.all_sunk():
                print("Your entire fleet has been sunk. AI wins!")
                return
```

board.py

```
class Board:
    SIZE = 6

    def __init__(self):
        self.ships = []
```

```
        self.shots = set()

    def place_ship(self, cells):
        cells = set(cells)

        if not cells:
            raise ValueError("A ship must contain at least one cell.")

        for r, c in cells:
            if not (0 <= r < self.SIZE and 0 <= c < self.SIZE):
                raise ValueError("Ship cell is outside the board.")

        for ship in self.ships:
            if cells & ship["cells"]:
                raise ValueError("Ships cannot overlap.")

        self.ships.append({
            "cells": cells,
            "hits": set()
        })

    def fire(self, pos):
        # Return (status, sunk_size).
        # status: "miss", "hit", "sunk", or "repeat".

        if pos in self.shots:
            return ("repeat", None)

        self.shots.add(pos)

        for ship in self.ships:
            if pos in ship["cells"]:
                ship["hits"].add(pos)

                if ship["hits"] == ship["cells"]:
                    return ("sunk", len(ship["cells"]))

                return ("hit", None)

        return ("miss", None)

    def remaining_cells(self):
        return sum(
            len(ship["cells"] - ship["hits"])
            for ship in self.ships
        )

    def all_sunk(self):
        return all(
            ship["hits"] == ship["cells"]
            for ship in self.ships
        )
```

ai.py

```
import random
```

```
class AI:
    def __init__(self, size=6):
        self.size = size
        self.tried = set()
        self.hits = set()

    def choose(self):
        # Never choose a coordinate that was already tried.
        options = [
            (r, c)
            for r in range(self.size)
            for c in range(self.size)
            if (r, c) not in self.tried
        ]

        if not options:
            return None

        # Prefer untried cells adjacent to previous hits.
        targets = set()

        for r, c in self.hits:
            for nr, nc in (
                (r - 1, c),
                (r + 1, c),
                (r, c - 1),
                (r, c + 1),
            ):
                if (
                    0 <= nr < self.size
                    and 0 <= nc < self.size
                    and (nr, nc) not in self.tried
                ):
                    targets.add((nr, nc))

        candidates = list(targets) if targets else options
        pos = random.choice(candidates)

        # Reserve the coordinate immediately to prevent repeats.
        self.tried.add(pos)
        return pos

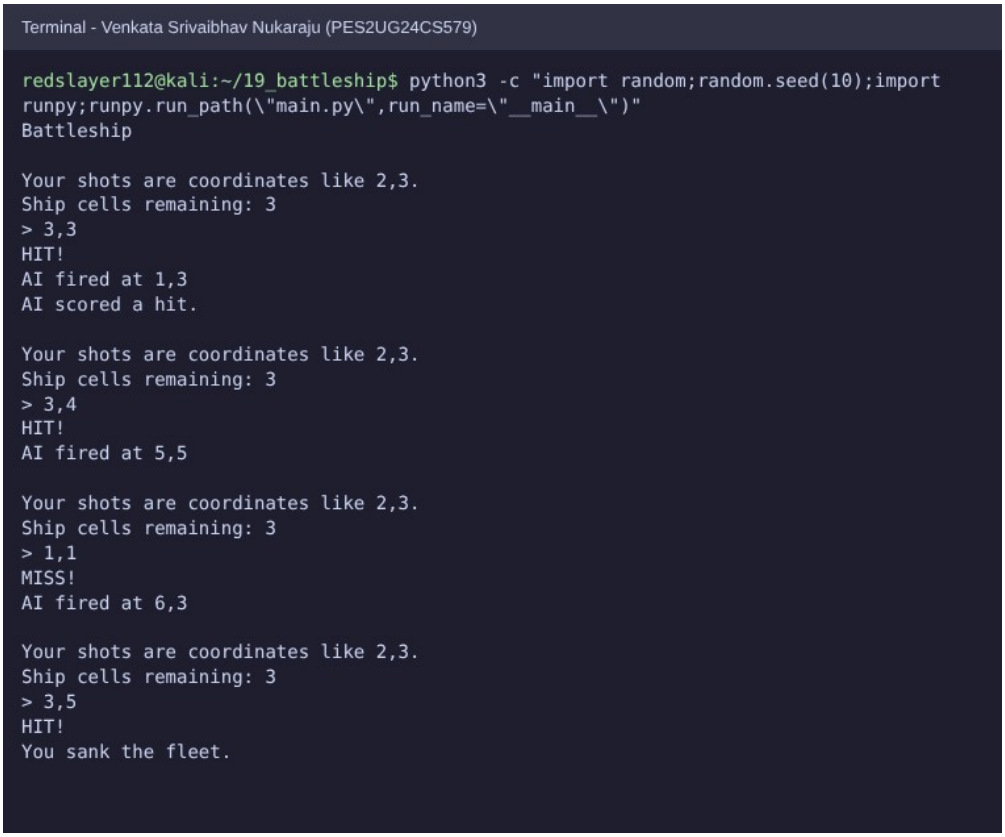
    def observe(self, pos, result):
        # Called only after the actual shot has been resolved.
        status, _ = result

        if status == "hit":
            self.hits.add(pos)
        elif status == "sunk":
            # The current hit completed a ship.
            self.hits.discard(pos)
```

4. Output Screenshots

Both runs use random seed 10, so the AI draws the same first cell. Before the fix, the AI shot at 1,3 counts as a hit on an empty cell. After the fix, the same shot is a real hit on the new fleet.

Screenshot 1: Before, false AI hit and stuck counter



Screenshot 2: After, full game with sunk ships and AI adjacent targeting

Terminal - Venkata Srivaibhav Nukaraju (PES2UG24CS579)

```
redslayer112@kali:~/19_battleship$ python3 -c "import random;random.seed(10);import
runpy;runpy.run_path(\"main.py\",run_name=\"__main__\")"
Battleship

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 7
> 4,1
You: HIT!
AI fired at 1,3
AI: HIT!

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 6
> 4,1
Already fired there.

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 6
> 9,9
Outside board.

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 6
> abc
Use row,col.

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 6
> 4,2
You sank a ship of size 2.
AI fired at 2,3
AI: MISS!

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 5
> 2,2
You: HIT!
AI fired at 1,4
AI: MISS!

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 4
> 2,3
You: HIT!
AI fired at 1,2
AI: HIT!

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 3
> 2,4
You sank a ship of size 3.
AI fired at 2,2
AI: MISS!

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 2
> 6,4
You: HIT!
AI fired at 1,1
AI sank a ship of size 3.

Your shots are coordinates like 2,3.
Enemy ship cells remaining: 1
> 6,5
You sank a ship of size 2.
You sank the entire enemy fleet. You win!
```

Screenshot 3: After, invalid input and quit

